Spectrometer Simulation Technical and User Documentation

Final Project Submission

October 2026

Abstract

This document describes the software architecture, physical model, numerical methods, configuration, execution workflow, outputs, validation strategy, and extension points of the spectrometer simulation contained in the `Final` directory. The software follows relativistic charged particles through prescribed electric and magnetic fields. It supports analytic Enge fringe-field models and numerically calculated fields, several time integrators, accurate boundary-crossing reconstruction, convergence studies, energy-conservation studies, and synthetic beam distributions for DLA- and TNSA-like experiments.

The Boolean blocks in `main.py` are intentional experiment switches. They allow individual studies to be isolated without maintaining several nearly identical driver files. They are not interpreted as unreachable or obsolete code in this documentation.

Project Scope

Purpose

The project simulates a charged-particle spectrometer in which particles enter through a pinhole and propagate under static electric and magnetic fields. The principal outputs are complete particle trajectories, exit positions, exit velocities, Lorentz factors, detector-impact distributions, and convergence or conservation diagnostics.

The implementation is suitable for the following studies:

- single-particle trajectories at prescribed kinetic energies;
- comparison of Euler, Runge–Kutta, and Boris integration methods;
- comparison of analytic and grid-based fields;
- the influence of Enge fringe fields and spectrometer geometry;
- angularly divergent monoenergetic beams;
- energy-distributed DLA-like beams;
- multi-species TNSA-like detector maps;
- time-step convergence and kinetic-energy conservation.

Model assumptions

The simulator treats particles as independent test particles. Their charge and current do not modify the prescribed fields; therefore space charge, self-fields, collisions, material interactions, radiation reaction, and collective plasma effects are outside the current model. The analytic and numeric field classes are two-dimensional in the (x, y) plane and return a three-component vector with a zero z component in the numeric model.

SI units are used internally. User-facing geometry and trajectory plots use millimetres, while particle energy is specified in MeV.

Directory and Module Structure

File or directory	Responsibility
`main.py`	Defines the baseline parameters and contains independently enabled experiment blocks.
`spectrometer/Experiment.py`	Converts units, calculates the initial relativistic state, time step, computational domain, and derived parameters.
`spectrometer/integrators.py`	Field selection, geometry termination, exit reconstruction, Euler, RK2, RK4, Boris, and collocated Boris pushers.
`spectrometer/analitic_fields.py`	Analytic magnetic and electric Enge fringe-field models.
`spectrometer/numeric_fields.py`	Multigrid Laplace solver, scalar potentials, staggered field extraction, linear interpolation, and natural-cubic interpolation.
`spectrometer/custom_spline_2D.py`	Project implementation of one- and two-dimensional natural cubic splines.
`spectrometer/physics.py`	Relativistic Lorentz acceleration, Boris magnetic rotation, and angular decomposition of velocity.
`spectrometer/simulators.py`	High-level trajectory, beam, convergence, energy, DLA, and TNSA workflows.
`spectrometer/geometry.py`	Three-dimensional drawing of the spectrometer, shield, and entrance aperture.
`spectrometer/plot_result.py`	Trajectory plotting helper.
`Figures/`	Saved trajectory, field, convergence, energy, DLA, and TNSA results.
`Fringe fields/`	Reference literature used for fringe-field modelling.

The file `fields.py` contains an older analytic-field implementation and plotting utility. The active integrator imports the analytic models from `analitic_fields.py`.

Physical Model

Relativistic equation of motion

The particle momentum is

$$\boldsymbol{p} = \gamma m \boldsymbol{v}, \qquad \gamma = \frac{1}{\sqrt{1 - |\boldsymbol{v}|^2/c^2}},$$

and the Lorentz force is

$$\frac{d\boldsymbol{p}}{dt} = q(\boldsymbol{E} + \boldsymbol{v} \times \boldsymbol{B}).$$

The velocity equation implemented by `get_lorentz_acceleration` is

$$\frac{d\boldsymbol{v}}{dt} = \frac{q}{\gamma m}\left[\boldsymbol{E} + \boldsymbol{v} \times \boldsymbol{B} - \frac{\boldsymbol{v}(\boldsymbol{E} \cdot \boldsymbol{v})}{c^2}\right].$$

The last term enforces the correct relativistic response to an electric field.

Energy-to-velocity conversion

The input `q_eng_MeV` is a three-component vector. Its norm determines the kinetic energy, while its direction determines the initial velocity direction. For kinetic energy K,

$$|\boldsymbol{v}_0| = c\sqrt{1 - \left(\frac{mc^2}{K + mc^2}\right)^2}.$$

Consequently, `[0, 3, 0]` represents a 3 MeV particle initially directed along positive y.

Coordinate system and geometry

The coordinate convention is:

- x: spectrometer width;
- y: longitudinal depth and nominal beam direction;
- z: spectrometer height.

The body occupies $|x| < w/2$, $0 < y < d$, and $|z| < h/2$. Before the entrance plane, the particle is restricted by the circular pinhole radius in both x and z. A particle is propagated only while it remains inside the relevant entrance or body limits and continues in the forward direction.

Analytic uniform-field exit solution

For the restricted uniform-field case, the project provides an analytic exit position for validation. With relativistic Larmor radius

$$R = \left|\frac{\gamma m v_{\perp}}{q B_x}\right|,$$

and vertical displacement h, the longitudinal detector coordinate is

$$y_{\text{exit}} = \sqrt{2Rh - h^2}.$$

The corresponding angle is

$$\theta = \cos^{-1}\left(\frac{R - h}{R}\right),$$

and the proper-time-like variable used in the code is

$$\tau = \frac{\theta m}{q B_x}.$$

The transverse displacement, including a uniform electric contribution, is then evaluated from the initial position, initial relativistic velocity, and constant acceleration in τ.

Field Models

Analytic Enge fringe fields

The analytic model uses the Enge function

$$F(\zeta) = \frac{1}{1 + \exp(S(\zeta))}, \qquad S(\zeta) = a_0 + a_1\zeta + a_2\zeta^2 + a_3\zeta^3.$$

The code forms a complex coordinate from $y + ix$ and obtains two transverse field components from the real and imaginary parts. Two coefficient sets are available through `sharp_edge`. When `yoke=0`, entrance and exit Enge factors are multiplied; when `yoke=1`, only the entrance transition is used. Setting `fringe=0` selects an abrupt uniform field inside the device and zero field before the entrance.

Numeric scalar-potential solution

The numeric model solves a two-dimensional Laplace problem on a rectangular grid. `MagneticPotential` and `ElectricPotential` override the base boundary conditions to represent pole faces or electrodes. The solution algorithm performs:

1. Gauss–Seidel smoothing on the finest grid;
2. residual calculation with second-order central differences;

3. restriction of the residual to successively coarser grids;
4. Gauss–Seidel solution of the correction equation;
5. bilinear prolongation of corrections toward the fine grid;
6. boundary-condition restoration and post-smoothing;
7. repetition until the residual threshold or iteration limit is reached.

Grid dimensions of the form $2^k + 1$ are appropriate for repeated coarsening. The solver prints its final residual and iteration count.

Field extraction on a staggered grid

The magnetic field is obtained from the magnetic scalar potential ψ:

$$B_x = -\mu_0 \frac{\partial \psi}{\partial x}, \qquad B_y = -\mu_0 \frac{\partial \psi}{\partial y}.$$

The electric field follows from $\boldsymbol{E} = -\nabla\phi$. Forward differences place x and y components at different half-grid locations. This staggered arrangement is preserved by supplying the appropriate coordinate axis to each interpolator.

Grid interpolation modes

The parameter `grid_interpulation` accepts the strings `"Linear"` and `"Spline"`.

Linear interpolation

Bilinear interpolation uses the four corners of the containing cell. It is inexpensive and continuous, but its first spatial derivatives change abruptly at cell boundaries. A high-order time integrator can therefore measure cell-crossing artefacts rather than its formal time order.

Natural cubic spline

The `"Spline"` option uses the project implementation in `custom_spline_2D.py`. It is constructed once from an axis pair and a field array:

```
spline = NaturalCubicSpline2D(x_axis, y_axis, field)
value = spline(x_particle, y_particle)
```

The field array must have shape `(len(y_axis), len(x_axis))`. A scalar query returns one interpolated component; array-valued coordinates return an array of component values.

For one dimension, the spline solves

$$A\boldsymbol{p}'' = \boldsymbol{b},$$

with natural boundary conditions $p''(x_0) = p''(x_N) = 0$. Each interval is stored in the same form as the original project derivation:

$$S_i(x) = \alpha_i(x - x_i)^3 + \beta_i(x - x_{i+1})^3 + \gamma_i(x - x_i) + \eta_i(x - x_{i+1}).$$

The two-dimensional spline first constructs the four x coefficients for each row and then splines those coefficients along y. Each cell therefore stores $4 \times 4 = 16$ coefficients. The coefficient tensor has shape

$$(N_y - 1, 4, N_x - 1, 4).$$

Construction is global, but each particle query identifies one cell and evaluates only its 16 coefficients. For a large grid the coefficient storage is approximately sixteen times the storage of one scalar field component.

Time Integration and Exit Events

Available integrators

Driver name	Formal order	Description
`Euler`	1	Explicit first-order update; primarily a baseline and error demonstration.
`RK2`	2	Midpoint Runge–Kutta method with midpoint field evaluation.
`RK4_Lin`	4 in the step	Classical four-stage RK4 followed by linear boundary reconstruction. Endpoint accuracy can be limited by the event interpolation.
`RK4_Herm`	4	Classical RK4 followed by cubic Hermite boundary reconstruction using position and velocity at both ends of the final step.
`Boris`	nominally 2	Electric half-kick, magnetic rotation, electric half-kick, using fields sampled at the current position.
`Boris_Coll`	2	Collocated variant that samples fields at a predicted half-step position and uses a centred position update.

The code chooses a time step from a magnetic time scale and `N_steps`:

$$\Delta t = \frac{T_{\text{cyclotron,code}}}{N_{\text{steps}}}.$$

Thus increasing `N_steps` refines time while leaving the field grid unchanged.

Linear exit reconstruction

`_exect_exist` calculates a fractional position between the last point inside and first point outside a boundary. Position is interpolated linearly. Velocity magnitude and direction

are treated separately before recombination, which reduces artificial changes of speed in magnetic-only motion.

Cubic Hermite exit reconstruction

`_hermite_exit_crossing` represents the last trajectory segment by

$$\boldsymbol{R}(s) = h_{00}\boldsymbol{R}_0 + h_{10}\Delta t\boldsymbol{v}_0 + h_{01}\boldsymbol{R}_1 + h_{11}\Delta t\boldsymbol{v}_1, \qquad 0 \le s \le 1,$$

where h_{00}, h_{10}, h_{01}, and h_{11} are the standard cubic Hermite basis functions. The crossed coordinate is solved by bisection. The full exit position is evaluated at the resulting fraction, and the tangent is obtained by differentiating the Hermite curve. The final velocity direction is the normalized tangent; its magnitude is interpolated independently between the two endpoint speeds. The exit Lorentz factor is recalculated.

This distinction is essential: RK4 can retain fourth-order endpoint behaviour only when both the right-hand side and the boundary event are reconstructed with sufficient smoothness and accuracy.

Configuration and Execution

Software requirements

The code directly imports:

- Python 3;
- NumPy;
- Matplotlib;
- SciPy;
- Joblib.

Example installation:

```
python -m pip install numpy matplotlib scipy joblib ipympl kaleido
```

Run from the directory containing `main.py`:

```
python main.py
```

On Windows, the `if __name__ == "__main__":` guard must be preserved because beam simulations use multiprocessing through Joblib.

Principal parameters

Parameter	Units/type	Meaning
`height_mm`	mm	Full spectrometer height in z.

Parameter	Units/type	Meaning
`width_mm`	mm	Full spectrometer width in x.
`depth_mm`	mm	Longitudinal length in y.
`shield_mm`	mm	Entrance shield length.
`pinhole_dia_mm`	mm	Entrance aperture diameter.
`yoke`	0 or 1	Controls geometry and whether the analytic model contains a separate exit fringe.
`R0_mm`	three-vector, mm	Initial position $[x_0, y_0, z_0]$.
`q_eng_MeV`	three-vector, MeV	Energy magnitude and initial direction.
`m_kg`	kg	Particle rest mass.
`q_C`	C	Signed particle charge.
`Bx0_T`	T	Nominal magnetic-field amplitude.
`Ex0_Vm`	V/m	Nominal electric-field amplitude.
`fringe`	0 or 1	Abrupt field or Enge/numeric fringe field.
`sharp_edge`	0 or 1	Selects one of the two Enge coefficient sets.
`solution`	string	`"Analitic field"` or `"Numeric Field"`.
`N_steps`	positive number	Time-resolution denominator.
`Nx_p`, `Ny_p`	integers	Potential-grid dimensions, preferably $2^k + 1$.
`grid_interpulation`	string	`"Linear"` or `"Spline"`.

Experiment switches in main.py

Each major study is enclosed in an `if True:` or `if False:` block. This is an intentional lightweight experiment-selection mechanism:

- set exactly the desired model-initialization block to `True`;
- set the desired experiment block or blocks to `True`;
- leave unrelated experiments at `False`;
- when reproducibility matters, start each block from a deep copy of `params_beckup`;
- remember that several `True` blocks execute sequentially and can open several figures or change shared parameter objects.

They should not be removed merely because static analysis labels them as constant conditions.

Selecting a numeric spline field

```
solution = "Numeric Field"
grid_interpulation = "Spline"
Nx_p = 2**7 + 1
Ny_p = 2**7 + 1
```

After changing any field-defining parameter, update the experiment and rebuild the selected field model:

```
params = experiment._update_params(params)
integrator._update_params(params)
integrator._update_solution(integrator.solution)
```

The rebuild is important for changes to grid size, domain, field amplitude, fringe configuration, geometry, or interpolation mode.

High-Level Simulation Workflows

Single and multiple trajectories

`basic_simulator` accepts an integrator, an integrator name, the experiment, and either one energy vector or an array of energy vectors. Each trajectory is run independently. Multiple particles use Joblib multiprocessing. The return values are nested collections of positions in millimetres, velocities in metres per second, and Lorentz factors.

Fully analytic mapping

`Fully_analitic` evaluates the restricted analytic exit relationship over an energy vector and species arrays. It is useful as a fast baseline and as a reference for uniform-field tests.

Convergence tests

`convergence_test` runs selected integrators over a sequence of `N_steps`, records exit positions, uses the finest result as a numerical reference, and fits a line in log–log space. For a reliable order estimate, use a geometric refinement series such as

$$N,\ 2N,\ 4N,\ 8N, \ldots$$

Energy-conservation study

`energy_vs_N_steps_for_different_integrators` compares the final Lorentz factor with its initial value for several methods and time resolutions. Magnetic-only motion provides the clearest conservation test, because a magnetic force changes direction but performs no work.

Angular and spectral beam models

`Non_collimataed_beams` samples azimuthal and elevation angles from exponential, Gaussian, uniform, or zero-width distributions. It rotates the initial velocity and optionally draws a two-dimensional impact histogram.

`real_beams` adds exponential, Gaussian, uniform, DLA gamma-distributed, or monoenergetic energy sampling. `TNSA` repeats this workflow for several mass, charge, and characteristic-energy values and combines the impact positions into one detector map.

For reproducible stochastic plots, set a NumPy random seed before sampling:

```
np.random.seed(12345)
```

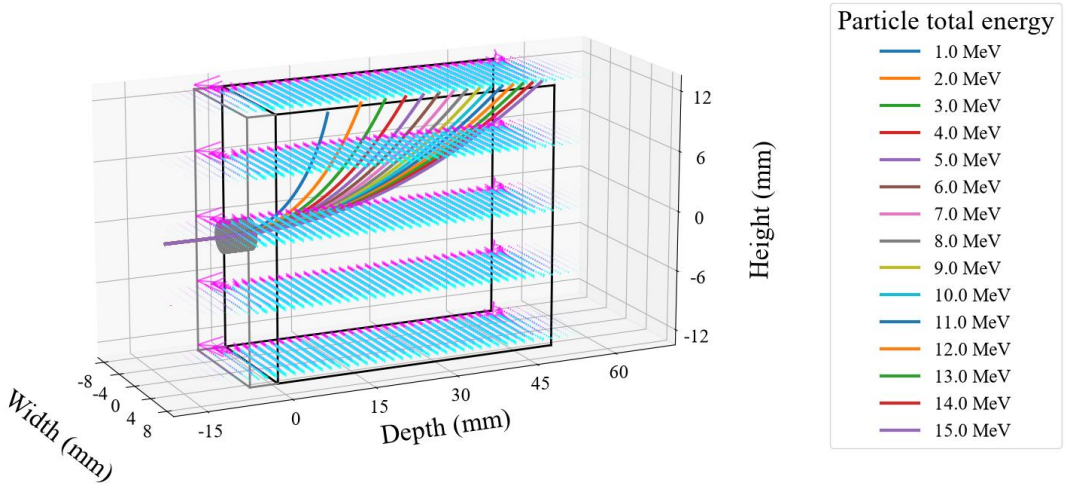
Outputs and Interpretation

Each trajectory integrator returns

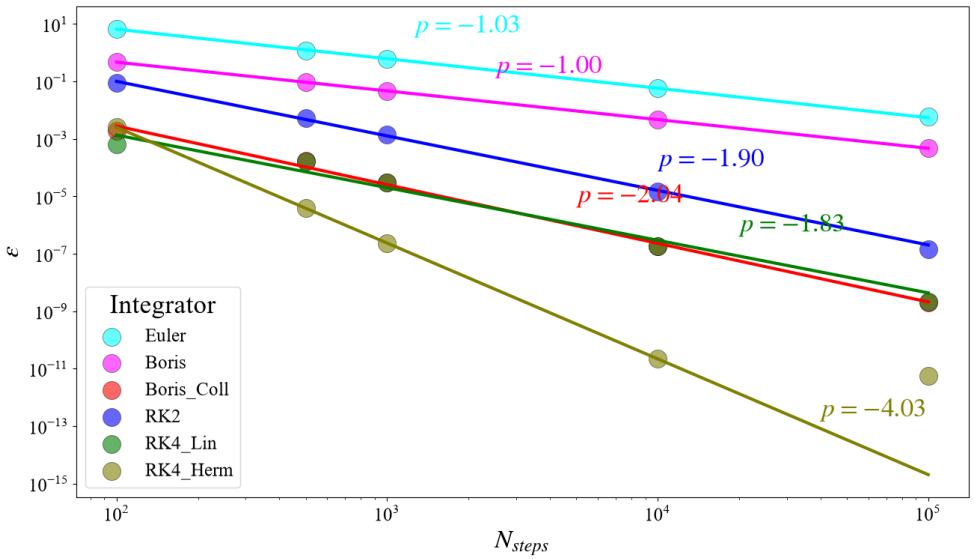
```
R_vec_mm, v_vec_m0s, gamma_vec
```

where the final sample is reconstructed on the boundary. Detector maps use the final (x, y) coordinates. Potential and field classes also provide visualization methods for scalar-potential maps, field magnitude, and streamlines.

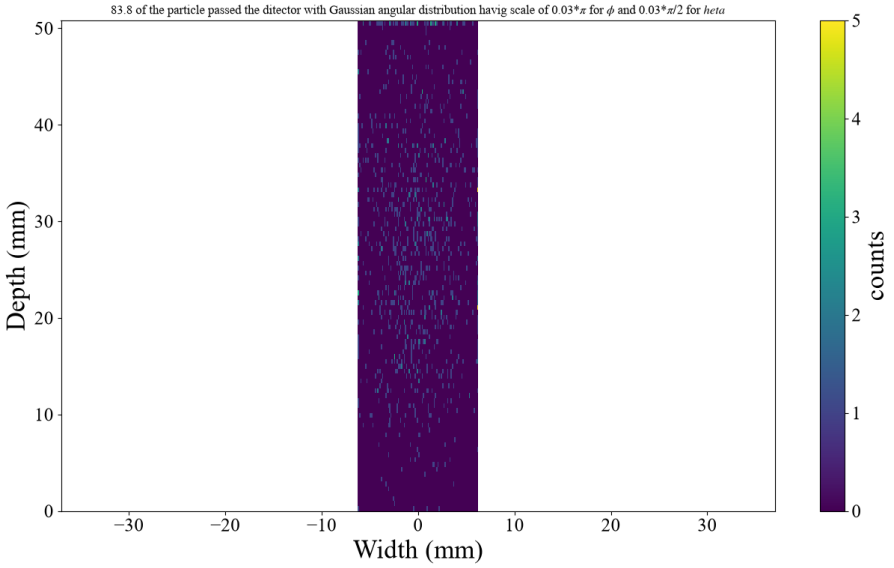
The `Figures` directory contains representative results. The following figures are included only when the files are available, so the document remains compilable if figures are moved.



Representative numeric field or potential result stored with the project.



Representative comparison of numerical integrators with the analytic solution.



Representative synthetic real-beam detector distribution.

Validation Strategy

A defensible validation sequence is:

1. verify energy-to-velocity conversion and γ reconstruction;
2. compare the uniform-field exit position with `_analitic_sol_vel2dist`;

3. verify expected time orders on the smooth analytic field;
4. compare linear and spline field interpolation on a fixed numeric grid;
5. verify magnetic-only speed or γ conservation;
6. refine the numeric potential grid separately from the time step;
7. repeat stochastic beam runs with a fixed seed;
8. record all geometry, species, field, grid, and time parameters with every final figure.

Expected theoretical guidance is Euler order one, midpoint RK2 order two, classical RK4 order four for a sufficiently smooth right-hand side, and Boris order two when the field sampling and staggering are consistent. Boundary reconstruction and field interpolation can limit the observed order even when the interior stepping formula has a higher formal order.

Implementation Notes and Limitations

- The spelling `Analitic field`, `grid_interpulation`, `_exect_exist`, and `RK4_Hermit` is part of the present program interface and must be used exactly unless the code is renamed consistently.
- The field model is built when `Integrators` is constructed or when `_update_solution` is called. Updating the parameter dictionary alone does not regenerate a numeric potential.
- The spline clips query coordinates to its tabulated domain. The numeric domain must therefore contain the complete trajectory region and fringe region of interest.
- Natural boundary conditions impose zero second derivative at the outer spline limits. The computational field domain should extend far enough that this is a reasonable approximation.
- Cubic splines may overshoot near genuine discontinuities. A physical Enge fringe is smooth, whereas an intentionally abrupt `fringe=0` field should not be interpreted as a smooth physical transition.
- The numeric field is based on a two-dimensional scalar-potential calculation and is not a general three-dimensional electromagnetic solver.
- Beam simulations can be memory- and CPU-intensive because integrator objects and field grids are copied into worker processes.
- Saved result figures are evidence of particular configurations; exact reproducibility additionally requires the parameter values, random seed, source version, and dependency versions.

Maintenance checks

When modifying the project, check the following paths together:

- both magnetic and electric interpolation paths;
- both analytic and numeric field construction branches;
- pinhole limits and body limits in every exit reconstruction method;
- scalar and array forms of energy input;
- single-process and multiprocessing execution;
- all enabled `if True` experiment blocks in `main.py`.

Quick-Start Checklist

1. Install NumPy, Matplotlib, SciPy, and Joblib.
2. Open `main.py` and set geometry, particle, and field parameters.
3. Enable one field-initialization block.
4. For numeric fields, select `"Linear"` or `"Spline"` and a $2^k + 1$ grid.
5. Enable the required experiment block and disable unrelated blocks.
6. Rebuild the field after changing field-defining parameters.
7. Run `python main.py` from the `Final` directory.
8. Save the figure together with its parameter set and random seed.
9. For convergence claims, inspect successive Richardson orders rather than relying only on one global fit.

Minimal Programmatic Example

```
import numpy as np
from spectrometer.Experiment import Experiment
from spectrometer.integrators import Integrators

experiment = Experiment(
    height_mm=26,
    width_mm=12.5,
    depth_mm=50.8,
    shield_mm=5,
    yoke=0,
    pinhole_dia_mm=3,
```

```
    R0_mm=np.array([0.0, -10.0, 0.0]),
    q_eng_MeV=np.array([0.0, 3.0, 0.0]),
    m_kg=9.109e-31,
    q_C=-1.602e-19,
    Bx0_T=0.5,
    Ex0_Vm=0.0,
    fringe=1,
    sharp_edge=1,
    N_steps=1600,
    CFL=0.05,
    solution="Numeric Field",
    Nx_p=129,
    Ny_p=129,
    grid_interpulation="Spline",
)

params = experiment._evaluate_exp_paramas()
integrator = Integrators(params)
R_mm, v_ms, gamma = integrator._RK4_Hermit()
print("Exit position [mm]:", R_mm[-1])
```

Natural Cubic Spline API

```
from spectrometer.custom_spline_2D import NaturalCubicSpline2D

# field.shape must be (len(y_axis), len(x_axis))
spline = NaturalCubicSpline2D(x_axis, y_axis, field)

# Scalar query -> scalar field-component value
field_at_particle = spline(x_particle, y_particle)

# Broadcast-compatible arrays -> array of field-component values
field_along_path = spline(x_positions, y_positions)
```

Internally, `P_tag_tag_fun` solves for natural-spline second derivatives and `Params_finder` converts them to $\alpha, \beta, \gamma, \eta$. These functions are exposed primarily to make the derivation transparent; normal simulation code should construct `NaturalCubicSpline2D` and call the resulting object.